

tpaHH: Two-Piece Asymmetric Hybrid Hazard Regression Models

Annotated Source Code and User Manual

Prepared for Ewnetu-github/tpaHH (originally `Topic3_sourcecode.R`)

24 August 2026

Contents

1	Introduction	1
1.1	What the toolkit does	1
1.2	How this document is organised	2
1.3	Requirements	2
2	Baseline Distributions and Link Functions	4
3	The TPA Distribution Family	5
3.1	Quick visual check	7
4	Fully Parametric Regression Models	8
5	Kernel Function and Semi-/Non-parametric Estimation	18
6	Post-Estimation Tools	27
7	Worked Example	31
8	Function Reference Table	34
9	Notes on the Original Code	35
10	Original Repository	35

1 Introduction

This document is a fully commented, reorganised rewrite of the original `Topic3_sourcecode.R` file from the GitHub repository `Ewnetu-github/tpaHH`. The underlying statistical logic of every function is **unchanged** from the original script — only the presentation has been rewritten: functions are grouped by purpose, every function is documented with a `roxygen2`-style header (`@param`, `@return`, `@details`), inline comments explain non-obvious steps, and a worked example at the end shows the full workflow end to end.

1.1 What the toolkit does

The code implements **hybrid hazard-based survival regression models** built from **two-piece asymmetric (TPA)** baseline distributions. In these models, a covariate can act on the survival time on two different scales at once:

- a **time-scale** (acceleration) effect, which stretches or compresses time itself (as in an Accelerated Failure Time, AFT, model), and

- a **hazard-scale** (relative-risk) effect, which multiplies the hazard up or down (as in a Proportional Hazards, PH, model).

Combining both gives the **Hybrid Hazard (HH)** model; letting only one of the two operate recovers the classical **AFT**, **PH**, **Accelerated Hazards (AH)**, or **General Hazards (GH)** models as special cases. The covariate effect on either scale can be modelled:

- **parametrically** (a linear index, e.g. $x^\top \beta$),
- **semi-parametrically** (part linear, part smooth), or
- **non-parametrically** (a fully smooth function estimated by local likelihood with a kernel).

Three baseline (error) distributions are supported for the two-piece family:

Code label	Baseline distribution
tpalnorm	TPA (log-)normal
tpallogis	TPA (log-)logistic
tpalLap	TPA (log-)Laplace

All three share the **same** log-link time transform $g(t) = \log(t)$, so the resulting failure-time distributions are, respectively, two-piece log-normal, log-logistic and log-Laplace.

1.2 How this document is organised

1. **Package setup** — the R packages the code depends on.
2. **Baseline distributions and link functions** — the building blocks.
3. **The TPA distribution family** — density, CDF, hazard, cumulative hazard, quantile function and random-number generator.
4. **Fully parametric regression models** — HH, GH, AFT, PH, AH.
5. **Kernel function and semi-/non-parametric estimation** — local-likelihood counterparts of the models above.
6. **Post-estimation tools** — prediction, distance measures, numerical integration, and confidence intervals.
7. **Worked example** — simulate data, fit a model, obtain standard errors, and plot a predicted survival curve.
8. **Function reference table** — a one-page lookup of every function.
9. **Notes on the original code** — a short, honest list of quirks/limits inherited from the original script, so users know what to check before relying on a given function.

1.3 Requirements

```
# Run once, interactively, before using this document.
# CRAN packages:
install.packages(c("nlme", "pracma"))

# Packages not currently on CRAN (or only as an archived version) --
# install from their GitHub mirrors instead:
install.packages("remotes")
remotes::install_github("cran/Deriv")      # symbolic differentiation
remotes::install_github("cran/GoFKernel")  # numerical inversion of a CDF

# QBAsyDist is only used in the original script for its expit() helper
# (the standard logistic/sigmoid function). This document defines expit()
# directly (see Section 2) so QBAsyDist is NOT required to run anything
# here. If you want the full QBAsyDist package for other purposes, see
```

```

# https://github.com/cran/QBAsyDist (it depends on several packages that
# are archived on CRAN, so plan for extra install steps).

suppressPackageStartupMessages({
  library(Deriv)      # symbolic differentiation, used by dtpa()
  library(GoFKernel) # inverse() : numerically inverts a CDF, used by qtpa()
  library(pracma)     # trapz(): trapezoidal numerical integration
  library(nlme)       # fdHess(): finite-difference Hessian, used for CIs
})

# expit() is the inverse-logit (standard logistic) function.
# The original script pulled this from the QBAsyDist package; because
# QBAsyDist is hard to install (see the note above) we define it directly.
# It is mathematically identical to QBAsyDist::expit().
expit <- function(x) 1 / (1 + exp(-x))

```

2 Baseline Distributions and Link Functions

Every TPA model is built by “gluing” two halves of a *reference* symmetric distribution together at a location parameter. The functions below provide the density (`f0_*`), CDF (`F0_*`) and quantile function (`QF_*`) of each of the three reference distributions used as building blocks, plus the link function `g()` that transforms time before the two-piece construction is applied.

```
#' Standard normal reference distribution
#' @param s numeric vector of evaluation points
#' @return f0_N: density; F0_N: CDF; QF_N: quantile function
#' @export
f0_N <- function(s) dnorm(s, mean = 0, sd = 1)           # density
F0_N <- function(s) pnorm(s, mean = 0, sd = 1)           # CDF
QF_N <- function(tau) qnorm(tau, mean = 0, sd = 1,        # quantile
                           lower.tail = TRUE, log.p = FALSE)

# Standard Laplace (double-exponential) reference distribution
#' @param s numeric vector of evaluation points
#' @param beta probability(-ies) in (0, 1) for the quantile function
#' @return f0_La: density; F0_La: CDF; QF_La: quantile function
#' @export
f0_La <- function(s) 0.5 * exp(-abs(s))                 # density
F0_La <- function(s) 0.5 + 0.5 * sign(s) * (1 - exp(-abs(s))) # CDF
QF_La <- function(beta) -sign(beta - 0.5) * log(1 - 2 * abs(beta - 0.5)) # quantile

# Standard logistic reference distribution
#' @param s numeric vector of evaluation points
#' @param tau probability(-ies) in (0, 1) for the quantile function
#' @return f0_Lo: density; F0_Lo: CDF; QF_Lo: quantile function
#' @export
f0_Lo <- function(s) exp(s) * (1 + exp(s))^-2           # density
F0_Lo <- function(s) 1 / (1 + exp(-s))                  # CDF
QF_Lo <- function(tau) log(tau / (1 - tau))              # quantile (logit)

# Link functions used to transform time before applying the two-piece
# construction. Only the log-link (glog) is exercised by the parametric
# and non-parametric estimators in this file; glogit/glogit.inv are
# provided for completeness / alternative link choices.
#' @param s numeric vector
#' @export
glog      <- function(s) log(s)           # time -> log(time)
glog.inv  <- function(s) exp(s)           # inverse of glog
glogit    <- function(s) log(exp(s) - 1)  # alternative link
glogit.inv <- function(s) log(exp(s) + 1)  # inverse of glogit
```

3 The TPA Distribution Family

These six functions define the two-piece asymmetric (TPA) distribution built from a symmetric reference distribution (f_0 , F_0 , QF) and a link function g . The construction follows Rubio & Steel-type two-piece distributions: a single location parameter η splits the real line into two halves, each half is a stretched/compressed copy of the reference distribution, and α controls how much probability mass sits in each half ($\alpha = 0.5$ recovers the symmetric case).

```
## Density of the TPA family
##
## @param t vector of quantiles (time)
## @param eta location parameter
## @param phi scale parameter (phi > 0)
## @param alpha skewness/index parameter in (0, 1)
## @param f0 density of the reference distribution, e.g. f0_N
## @param g link function applied to time, e.g. glog
## @param log logical; if TRUE, the log-density is returned
## @return numeric vector of (log-)density values, same length as t
## @export
dtpa <- function(t, eta, phi, alpha, f0, g, log = FALSE) {
  # g.prime is the derivative of the link function, obtained symbolically
  # (this is why the Deriv package is required). For g = glog = log(t),
  # g.prime(t) = 1/t.
  g.prime <- Deriv::Deriv(g)

  # Common multiplicative term shared by both pieces of the density.
  com <- 2 * alpha * (1 - alpha) * g.prime(t) / phi

  # Left piece (t < eta) uses (1 - alpha) as the scale factor;
  # right piece (t >= eta) uses alpha. This is what makes the
  # distribution asymmetric around eta.
  den <- ifelse(t < eta,
               com * f0((1 - alpha) * (g(eta) - g(t)) / phi),
               com * f0(alpha * (g(t) - g(eta)) / phi))

  if (log) return(log(den)) else return(den)
}

## Cumulative distribution function (CDF) of the TPA family
## @param q vector of quantiles (time)
## @param eta, phi, alpha TPA parameters (see dtpa)
## @param F0 CDF of the reference distribution, e.g. F0_N
## @param g link function, e.g. glog
## @param lower.tail logical; if FALSE, 1 - CDF (the survival function) is returned
## @return numeric vector of (survival) probabilities
## @export
ptpa <- function(q, eta, phi, alpha, F0, g, lower.tail = TRUE) {
  p <- ifelse(q < eta,
              2 * alpha * F0((1 - alpha) * (g(q) - g(eta)) / phi),
              2 * alpha - 1 + 2 * (1 - alpha) * F0(alpha * (g(q) - g(eta)) / phi))
  if (lower.tail) return(p) else return(1 - p)
}

## Hazard function of the TPA family: h(t) = f(t) / S(t)
## @param t vector of quantiles (time)
```

```

#' @param eta,phi,alpha TPA parameters
#' @param f0 reference density
#' @param F0 reference CDF
#' @param g link function
#' @param log logical; if TRUE, the log-hazard is returned
#' @export
htpa <- function(t, eta, phi, alpha, f0, F0, g, log = FALSE) {
  dens <- dtpa(t, eta, phi, alpha, f0, g, log = FALSE)
  surv <- ptpa(t, eta, phi, alpha, F0, g, lower.tail = FALSE)
  h <- dens / surv
  if (log) return(log(h)) else return(h)
}

```

```

#' Cumulative hazard function of the TPA family:  $H(t) = -\log(S(t))$ 
#' @param t vector of quantiles (time)
#' @param eta,phi,alpha TPA parameters
#' @param F0 reference CDF
#' @param g link function
#' @param log logical; if TRUE, log(H(t)) is returned
#' @export
Htpa <- function(t, eta, phi, alpha, F0, g, log = FALSE) {
  cmh <- -log(ptpa(t, eta, phi, alpha, F0, g, lower.tail = FALSE))
  if (log) return(log(cmh)) else return(cmh)
}

```

```

#' Quantile function of the TPA family
#'
#' @param tau vector of probabilities in (0, 1)
#' @param eta,phi,alpha TPA parameters
#' @param F0 reference CDF
#' @param g link function
#' @param g.inv inverse link function; if NULL it is obtained numerically
#'   via GoFKernel::inverse()
#' @param QF analytic quantile function of the reference distribution
#'   (e.g. QF_N); if NULL it is obtained numerically via GoFKernel::inverse()
#' @return numeric vector of quantiles, same length as tau
#' @export
qtpa <- function(tau, eta, phi, alpha, F0, g, g.inv = NULL, QF = NULL) {
  # Fall back to numerical inversion when analytic forms are not supplied.
  if (is.null(QF)) {
    QF <- GoFKernel::inverse(F0, lower = -Inf, upper = Inf)
  }
  if (is.null(g.inv)) {
    g.inv <- GoFKernel::inverse(g, lower = 0, upper = Inf)
  }

  q <- vector()
  for (i in 1:length(tau)) {
    # Ca is the standardised quantile on the reference scale, computed
    # piecewise depending on which side of alpha the target probability
    # falls on.
    Ca <- ifelse(tau[i] < alpha,
                  (1 / (1 - alpha)) * QF(tau[i] / (2 * alpha)),
                  (1 / alpha) * QF((1 + tau[i] - 2 * alpha) / (2 * (1 - alpha))))
  }
}

```

```

    q[i] <- g.inv(g(eta) + phi * Ca)
  }
  return(q)
}

#' Random number generation from the TPA family (inverse-CDF method)
#' @param n number of draws
#' @param eta, phi, alpha TPA parameters
#' @param F0 reference CDF
#' @param g link function
#' @param QF quantile function of the reference distribution
#' @return numeric vector of n simulated survival times
#' @export
rtpa <- function(n, eta, phi, alpha, F0, g, QF) {
  u <- runif(n, min = 0, max = 1)
  r <- vector()
  for (i in 1:n) {
    r[i] <- qtpa(u[i], eta, phi, alpha, F0, g, QF)
  }
  return(r)
}

```

3.1 Quick visual check

The chunk below plots the TPA-normal density and hazard function for a skewed configuration ($\alpha \neq 0.5$), as a sanity check that the functions above behave as expected.

```

tt <- seq(0.05, 6, length.out = 400)
par(mfrow = c(1, 2))
plot(tt, dtpa(tt, eta = 1, phi = 0.6, alpha = 0.35, f0 = f0_N, g = glog),
     type = "l", xlab = "t", ylab = "density", main = "dtpa()")
plot(tt, htpa(tt, eta = 1, phi = 0.6, alpha = 0.35, f0 = f0_N, F0 = F0_N, g = glog),
     type = "l", xlab = "t", ylab = "hazard", main = "htpa()")

par(mfrow = c(1, 1))

```

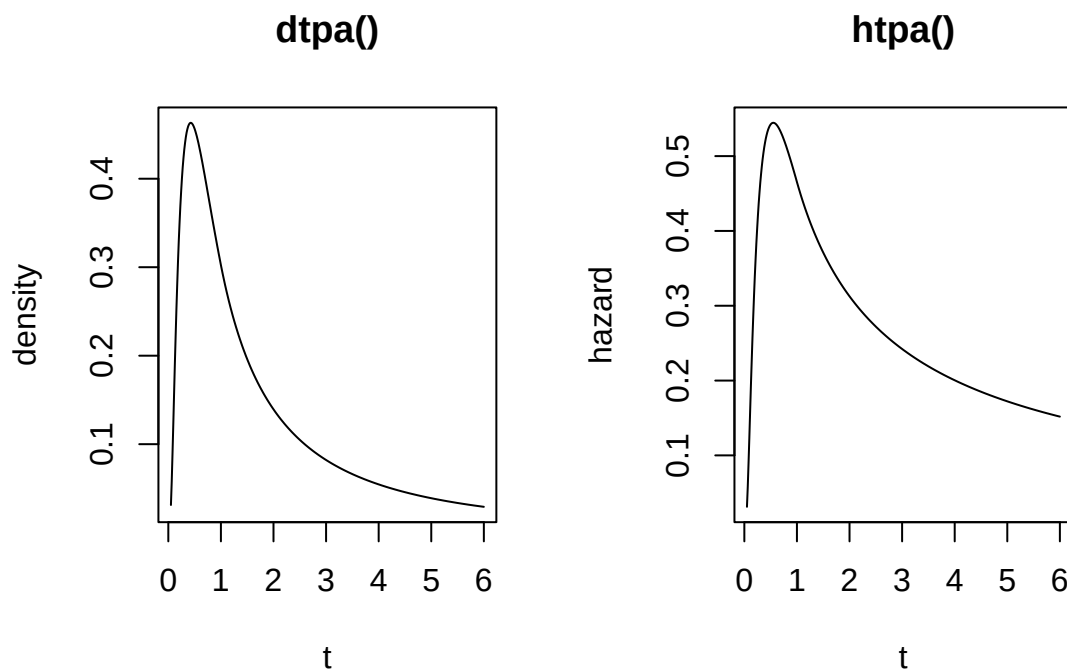


Figure 1: TPA-normal density and hazard for $\eta=1$, $\phi=0.6$, $\alpha=0.35$

4 Fully Parametric Regression Models

Each function in this section maximises a **negative log-likelihood** built from `htpa()`/`Htpa()` by a numerical optimiser (`optim()` or `nlminb()`), returning both the optimiser output and the negative log-likelihood function itself (useful later for standard errors, see `Conf.Int.FD()`).

All five functions share the same parameter-vector layout for the baseline distribution: `par[1] = log(eta)`, `par[2] = log(phi)`, `par[3] = logit(alpha)` (so `eta`, `phi` stay positive and `alpha` stays in (0, 1) during unconstrained optimisation), followed by regression coefficients.

Function	Model	Covariate role(s)
<code>parHH_mle()</code>	HH	<code>x1</code> on the time scale, <code>x2</code> on the hazard scale (both linear or <code>x2</code> profiled out if <code>psi2.hat</code> is supplied)
<code>parGH_mle()</code>	GH	same design matrix <code>x</code> used on both scales, with separate coefficient vectors
<code>parAFT_mle()</code>	AFT	<code>x</code> on the time scale only (pure acceleration)
<code>parPH_mle()</code>	PH	<code>x</code> on the hazard scale only (pure proportional hazards)
<code>parAH_mle()</code>	AH	<code>x</code> on the time scale, with the hazard-scale effect being the <i>negative</i> of the same linear predictor (accelerated hazards)

```
#' Maximum likelihood estimation for the parametric Hybrid Hazard (HH) model
#'
```

```
#' The HH model allows a covariate to act simultaneously on the time scale
#' (through x1) and the hazard scale (through x2). Semi-parametric use is
#' supported: pass a pre-estimated `psi2.hat` (a numeric vector, one value
#' per observation) to hold the hazard-scale linear predictor fixed while
```



```

#' profiling over the remaining parameters -- this is how the code is later
#' combined with the non-parametric estimators in Section 5.
#'
#' @param init numeric vector of starting values for the optimiser
#' @param y observed (possibly censored) survival times
#' @param delta event indicator (1 = event, 0 = censored)
#' @param x1 design matrix for the time-scale covariates (n x d1)
#' @param x2 design matrix for the hazard-scale covariates (n x d2)
#' @param dist baseline distribution: "tpalnorm", "tpallogis" or "tpalLap"
#' @param psi2.hat optional fixed hazard-scale linear predictor (length n);
#'   if supplied, the x2/beta2 block is not estimated
#' @param method optimiser: any `optim()` method, or "nlminb"
#' @param maxit maximum number of iterations (default 10000)
#' @return a list with `OPT` (raw optimiser output) and `nloglik`
#'   (the negative log-likelihood function used, for later reuse e.g. by
#'   `Conf.Int.FD()`)
#' @export
parHH_mle <- function(init, y, delta, x1, x2, dist, psi2.hat = NULL,
                      method = "Nelder-Mead", maxit = 10000, ...) {
  y <- as.vector(y)
  x1 <- as.matrix(x1)
  x2 <- as.matrix(x2)
  d1 <- dim(x1)[2]
  d2 <- dim(x2)[2]

  # --- TPA log-normal HH model -----
  if (dist == "tpalnorm") {
    nlog.lik <- function(par) {
      if (anyNA(par)) return(Inf)
      # Baseline parameters, transformed back to their natural scale.
      eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
      beta1 <- par[4:(3 + d1)] # time-scale regression coefficients
      psi.x1 <- as.vector(x1 %*% beta1)
      exp.psi.x1 <- exp(psi.x1)

      # If psi2.hat is not supplied, estimate beta2 along with everything
      # else; otherwise treat the hazard-scale linear predictor as known
      # (profiled out) -- this is the hook used for semi-parametric fits.
      if (is.null(psi2.hat)) {
        beta2 <- par[(4 + d1):(3 + d1 + d2)]
        psi.x2 <- as.vector(x2 %*% beta2)
      } else {
        psi.x2 <- psi2.hat
      }

      exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

      # Log-hazard contribution (evaluated at the time-rescaled argument
      # y * exp(psi.x1)) and cumulative-hazard contribution, combined into
      # the negative log-likelihood of a right-censored sample.
      lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_N, F0_N,
                    glog, log = TRUE) + psi.x2
      chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_N,

```

```

        glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

# --- TPA log-logistic HH model -----
if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta1 <- par[4:(3 + d1)]
    psi.x1 <- as.vector(x1 %*% beta1)
    exp.psi.x1 <- exp(psi.x1)

    if (is.null(psi2.hat)) {
      beta2 <- par[(4 + d1):(3 + d1 + d2)]
      psi.x2 <- as.vector(x2 %*% beta2)
    } else {
      psi.x2 <- psi2.hat
    }

    exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_Lo, F0_Lo,
      glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_Lo,
      glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

# --- TPA log-Laplace HH model -----
if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta1 <- par[4:(3 + d1)]
    psi.x1 <- as.vector(x1 %*% beta1)
    exp.psi.x1 <- exp(psi.x1)

    if (is.null(psi2.hat)) {
      beta2 <- par[(4 + d1):(3 + d1 + d2)]
      psi.x2 <- as.vector(x2 %*% beta2)
    } else {
      psi.x2 <- psi2.hat
    }
  }
}

```

```

exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_La, F0_La,
             glog, log = TRUE) + psi.x2
chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_La,
             glog, log = FALSE) * exp.psi.dif
lhaz0[is.infinite(lhaz0)] <- NA
chaz0[is.infinite(chaz0)] <- NA
val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
return(val)
}
}

# --- Optimisation -----
if (method != "nlminb") {
  OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
}
if (method == "nlminb") {
  OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
}
OUT <- list(OPT = OPT, nloglik = nlog.lik)
return(OUT)
}

#' Maximum likelihood estimation for the parametric General Hazards (GH) model
#'
#' The GH model uses the *same* design matrix `x` for both the time scale
#' and the hazard scale, but with two separate coefficient vectors
#' (`beta1`, `beta2`). This differs from `parHH_mle()`, which allows two
#' different design matrices `x1`/`x2`.
#'
#' @inheritParams parHH_mle
#' @param x common design matrix for both scales (n x d)
#' @export
parGH_mle <- function(init, y, delta, x, dist, method = "Nelder-Mead",
                      maxit = 10000, ...) {
  y <- as.vector(y)
  delta <- as.vector(delta)
  x <- as.matrix(x)
  d <- dim(x)[2]

  if (dist == "tpalnorm") {
    nlog.lik <- function(par) {
      if (anyNA(par)) return(Inf)
      eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
      beta1 <- par[4:(3 + d)] # time-scale coefficients
      beta2 <- par[(4 + d):(3 + 2 * d)] # hazard-scale coefficients

      psi.x1 <- as.vector(x %*% beta1)
      psi.x2 <- as.vector(x %*% beta2)

      exp.psi.x1 <- exp(psi.x1)
      exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

```

```

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta1 <- par[4:(3 + d)]
    beta2 <- par[(4 + d):(3 + 2 * d)]
    psi.x1 <- as.vector(x %>% beta1)
    psi.x2 <- as.vector(x %>% beta2)

    exp.psi.x1 <- exp(psi.x1)
    exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta1 <- par[4:(3 + d)]
    beta2 <- par[(4 + d):(3 + 2 * d)]

    psi.x1 <- as.vector(x %>% beta1)
    psi.x2 <- as.vector(x %>% beta2)

    exp.psi.x1 <- exp(psi.x1)
    exp.psi.dif <- as.vector(exp(-psi.x1 + psi.x2))

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_La, glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)

```

```

if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- list(OPT = OPT, nloglik = nlog.lik)
return(OUT)
}

#' Maximum likelihood estimation for the parametric Accelerated Failure
#' Time (AFT) model (time-scale covariate effect only)
#'
#' @inheritParams parGH_mle
#' @param psihat optional fixed extra linear predictor (length n), added to
#' the estimated  $x \% \beta$  term; defaults to a vector of zeros
#' @export
parAFT_mle <- function(init, y, delta, x, dist, method = "Nelder-Mead",
                      psihat = NULL, maxit = 10000, ...) {
  y <- as.vector(y)
  x <- as.matrix(x)
  d <- dim(x)[2]
  if (is.null(psihat)) {
    psi.x2 <- as.matrix(rep(0, length(y)))
  } else {
    psi.x2 <- psihat
  }

  if (dist == "tpalnorm") {
    nlog.lik <- function(par) {
      if (anyNA(par)) return(Inf)
      eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
      beta <- par[4:(3 + d)]
      psi.x1 <- as.vector(x \% as.vector(beta))
      psi.x <- psi.x1 + psi.x2
      exp.psi.x <- exp(psi.x)

      # Note: in the pure AFT model the cumulative hazard is NOT rescaled
      # by exp.psi.x (unlike PH/HH) -- only the argument of the hazard is
      # rescaled by time (y * exp.psi.x). This is what makes it an
      # accelerated-*time* rather than a proportional-*hazard* model.
      lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x
      chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_N, glog, log = FALSE)
      lhaz0[is.infinite(lhaz0)] <- NA
      chaz0[is.infinite(chaz0)] <- NA
      val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
      return(val)
    }
  }

  if (dist == "tpallogis") {
    nlog.lik <- function(par) {
      if (anyNA(par)) return(Inf)
      eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
      beta <- par[4:(3 + d)]
      psi.x1 <- as.vector(x \% as.vector(beta))
      psi.x <- psi.x1 + psi.x2
      exp.psi.x <- exp(psi.x)

```

```

    lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x
    chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta <- par[4:(3 + d)]
    psi.x1 <- as.vector(x %*% as.vector(beta))
    psi.x <- psi.x1 + psi.x2
    exp.psi.x <- exp(psi.x)

    lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE) + psi.x
    chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_La, glog, log = FALSE)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- list(OPT = OPT, nloglik = nlog.lik)
return(OUT)
}

#' Maximum likelihood estimation for the parametric Proportional Hazards
#' (PH) model (hazard-scale covariate effect only)
#'
#' Also usable in a semi-parametric setting: pass a pre-estimated
#' `psihat` to hold the linear predictor fixed.
#'
#' @inheritParams parAFT_mle
#' @export
parPH_mle <- function(init, y, delta, x, dist, psihat = NULL,
  method = "Nelder-Mead", maxit = 10000, ...) {
  y <- as.vector(y)
  x <- as.matrix(x)
  d <- dim(x)[2]
  if (is.null(psihat)) {
    psi.x2 <- as.matrix(rep(0, length(y)))
  } else {
    psi.x2 <- psihat
  }

  if (dist == "tpalnrm") {
    nlog.lik <- function(par) {

```

```

    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta <- par[4:(3 + d)]
    psi.x1 <- as.vector(x %*% as.vector(beta))
    psi.x <- psi.x1 + psi.x2
    exp.psi.x <- exp(psi.x)

    # PH model: time itself is NOT rescaled (unlike AFT); only the
    # hazard is multiplied by exp(psi.x) -- the defining property of a
    # proportional-hazards model.
    lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x
    chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp.psi.x
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    # NOTE (kept as in the original script): unlike the tpalnorm branch
    # above, this branch and the tpalLap branch below check `psihat`
    # again here and re-derive psi.x from it, rather than reusing the
    # psi.x2 computed at the top of the function. Functionally
    # equivalent, but worth knowing if you are diffing against your own
    # modifications.
    if (is.null(psihat)) {
      beta <- par[4:(3 + d)]
      psi.x <- as.vector(x %*% beta)
      exp.psi.x <- exp(psi.x)
    } else {
      psi.x <- psihat
      exp.psi.x <- exp(psihat)
    }

    lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x
    chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp.psi.x
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    if (is.null(psihat)) {
      beta <- par[4:(3 + d)]

```

```

    psi.x <- as.vector(x %*% beta)
    exp.psi.x <- exp(psi.x)
  } else {
    psi.x <- psihat
    exp.psi.x <- exp(psihat)
  }

  lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE) + psi.x
  chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_La, glog, log = FALSE) * exp.psi.x
  lhaz0[is.infinite(lhaz0)] <- NA
  chaz0[is.infinite(chaz0)] <- NA
  val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
  return(val)
}
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- list(OPT = OPT, nloglik = nlog.lik)
return(OUT)
}

#' Maximum likelihood estimation for the parametric Accelerated Hazards
#' (AH) model
#'
#' The AH model rescales time by exp(psi.x) (like AFT) AND divides the
#' cumulative hazard contribution by exp(psi.x) (i.e. multiplies by
#' exp(-psi.x)). This combination is what characterises the accelerated
#' hazards model, distinct from AFT and PH.
#'
#' @inheritParams parPH_mle
#' @export
parAH_mle <- function(init, y, delta, x, dist, psihat = NULL,
                      method = "Nelder-Mead", maxit = 10000, ...) {
  y <- as.vector(y)
  x <- as.matrix(x)
  d <- dim(x)[2]
  if (is.null(psihat)) {
    psi.x2 <- as.matrix(rep(0, length(y)))
  } else {
    psi.x2 <- psihat
  }

  if (dist == "tpalnorm") {
    nlog.lik <- function(par) {
      if (anyNA(par)) return(Inf)
      eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
      beta <- par[4:(3 + d)]
      psi.x1 <- as.vector(x %*% beta)
      psi.x <- psi.x1 + psi.x2
      exp.psi.x <- exp(psi.x)

      lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE)
      chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp(-psi.x)

```



```

    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta <- par[4:(3 + d)]
    psi.x1 <- as.vector(x %*% beta)
    psi.x <- psi.x1 + psi.x2
    exp.psi.x <- exp(psi.x)

    lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE)
    chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp(-psi.x)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(par[1]); phi0 <- exp(par[2]); alpha0 <- expit(par[3])
    beta <- par[4:(3 + d)]
    psi.x1 <- as.vector(x %*% beta)
    psi.x <- psi.x1 + psi.x2
    exp.psi.x <- exp(psi.x)

    lhaz0 <- htpa(y * exp.psi.x, eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE)
    chaz0 <- Htpa(y * exp.psi.x, eta0, phi0, alpha0, F0_La, glog, log = FALSE) * exp(-psi.x)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0, na.rm = TRUE) + sum(chaz0, na.rm = TRUE)
    return(val)
  }
}

if (method != "nllminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nllminb") OPT <- nllminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- list(OPT = OPT, nloglik = nlog.lik)
return(OUT)
}

```

5 Kernel Function and Semi-/Non-parametric Estimation

The functions in this section implement **local-likelihood** estimation of a smooth covariate effect, i.e. the non-parametric building block used when part of a model's linear predictor is left unspecified. Estimation proceeds by:

1. picking a **kernel** (weight function) `ker()`,
2. centring the covariate of interest on a grid point `x0` and weighting nearby observations more heavily using bandwidth `h`, and
3. locally approximating the unknown smooth function by a second-order polynomial (a *local quadratic* fit), whose coefficients are estimated by maximising the kernel-weighted log-likelihood.

Each `nonpar*_mle()` function below is the local-likelihood, kernel-weighted counterpart of the corresponding `par*_mle()` function in Section 4, and each optionally “profiles out” a parametric part of the covariate effect (pass its index via `x21.index/x11.index` and its fitted coefficients via `beta2.hat/beta1.hat`) – this is what allows a genuinely **semi-parametric** model (part linear, part smooth) to be fitted.

```
#' Kernel weight functions used for local-likelihood smoothing
#' @param ktype one of "uniform", "Gaussian", "Epan" (Epanechnikov,
#'   the default used throughout), "biweight", "triweight", "cosine"
#' @param x standardised distance (x0 - xi) / h
#' @return kernel weights, same length as x
#' @export
ker <- function(ktype, x) {
  if (ktype == "uniform") {
    result <- dunif(x, -1, 1)
  }
  if (ktype == "Gaussian") {
    result <- dnorm(x)
  } else if (ktype == "Epan") {
    result <- ifelse(abs(x) <= 1, 3 / 4 * (1 - x^2), 0)
  } else if (ktype == "biweight") {
    result <- ifelse(abs(x) <= 1, 15 / 16 * (1 - x^2)^2, 0)
  } else if (ktype == "triweight") {
    result <- ifelse(abs(x) <= 1, 35 / 32 * (1 - x^2)^3, 0)
  } else if (ktype == "cosine") {
    result <- ifelse(abs(x) <= 1, pi / 4 * cos(pi / 2 * x), 0)
  }
  return(result)
}

#' Local-likelihood (semi-/non-parametric) estimation of the hazard-scale
#' smooth term in the HH model
#'
#' @param init optional starting values; if NULL, a degree-2 polynomial OLS
#'   fit on the (uncensored) data is used to construct starting values
#' @param y,delta observed times and event indicator
#' @param x1 time-scale design matrix (parametric part, already estimated
#'   via beta1.hat)
#' @param x2 hazard-scale covariate matrix; may combine a parametric block
#'   (columns given by `x21.index`, coefficients `beta2.hat`) and a
#'   non-parametric block (the remaining columns)
#' @param x21.index column indices of x2 that are modelled parametrically;
#'   NULL if the entire hazard-scale effect is non-parametric
#' @param theta.hat fitted (transformed) baseline parameters, i.e.
#'   `c(log(eta), log(phi), logit(alpha))`
```

```

#' @param beta1.hat fitted time-scale regression coefficients
#' @param beta2.hat fitted coefficients for the parametric block of x2
#'   (required if x21.index is supplied)
#' @param dist baseline distribution: "tpalnorm" or "tpallogis" (the
#'   log-Laplace case is not implemented for this estimator in the
#'   original code)
#' @param x0 the local grid point at which the smooth function is estimated
#' @param h bandwidth (> 0)
#' @param ktype kernel type, see `ker()`; defaults to "Epan"
#' @param method optimiser, "Nelder-Mead" (default) or "nlminb"
#' @param maxit maximum iterations
#' @return the fitted local polynomial coefficients at x0 (intercept,
#'   linear, quadratic term); the function value at x0 is the first element
#' @export
nonparHH_mle <- function(init = NULL, y, delta, x1, x2, x21.index = NULL,
                        theta.hat, beta1.hat, beta2.hat = NULL, dist,
                        x0, h, ktype = "Epan", method = "Nelder-Mead", maxit = 10000, ...) {
  if (is.na(h) || h <= 0L) stop("invalid 'bandwidth value.' ")
  if (is.null(colnames(x2))) stop("give a column names for the 'covariates'")

  y <- as.vector(y)
  delta <- as.vector(delta)
  x1 <- as.matrix(x1)
  d1 <- dim(x1)[2]
  n <- length(y)

  # Split x2 into a parametric block (x21) and the non-parametric block
  # (x22) that is actually being smoothed at this call.
  if (!is.null(x21.index) | !is.null(beta2.hat)) {
    x21 <- as.matrix(x2[, x21.index])
    d21 <- ncol(x21)
    x22 <- as.matrix(x2[, -c(1:d21)])
    d22 <- ncol(x22)
    psi.x21 <- as.vector(x21 %*% beta2.hat)
  } else {
    psi.x21 <- rep(0, n)
    x22 <- as.matrix(x2)
    d22 <- dim(x22)[2]
  }

  # Centre the non-parametric covariate(s) on the local point x0 and build
  # kernel weights, then a local design matrix up to the quadratic term.
  newx0 <- x22 - t(matrix(rep(x0, d22 * n), d22, n))
  ker.h <- as.vector(apply(newx0 / h, MARGIN = 2, FUN = ker, ktype = ktype) / h)
  newx <- cbind(1, newx0, newx0^2) # local quadratic (up to second derivative)

  if (is.null(init)) {
    fit.poly <- lm(y[delta == 1] ~ poly(newx0[delta == 1], degree = 2))
    init <- as.numeric(fit.poly$coefficients)
  }

  if (dist == "tpalnorm") {
    nlog.lik <- function(par) {

```

```

    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b2 <- par
    psi.x22 <- as.vector(newx %*% b2)

    psi.x2 <- psi.x21 + psi.x22
    psi.x1 <- as.vector(x1 %*% beta1.hat)
    exp.psi.x1 <- exp(psi.x1)
    exp.psi.dif <- exp(-psi.x1 + psi.x2)

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    # Kernel-weighted (local) negative log-likelihood.
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b2 <- par
    psi.x22 <- as.vector(newx %*% b2)

    psi.x2 <- psi.x21 + psi.x22
    psi.x1 <- as.vector(x1 %*% beta1.hat)
    exp.psi.x1 <- exp(psi.x1)
    exp.psi.dif <- exp(-psi.x1 + psi.x2)

    lhaz0 <- htpa(y * exp.psi.x1, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y * exp.psi.x1, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp.psi.dif
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- OPT$par
return(OUT)
}

#' Local-likelihood estimation of the hazard-scale smooth term in the PH
#' model (no time-scale rescaling)
#' @inheritParams nonparHH_mle
#' @param x2 as in nonparHH_mle
#' @export
nonparPH_mle <- function(init = NULL, y, delta, x2, x21.index = NULL,
  theta.hat, beta2.hat = NULL, dist,

```

```

      x0, h, ktype = "Epan", method = "Nelder-Mead", maxit = 10000, ...) {
if (is.na(h) || h <= 0L) stop("invalid 'bandwidth value.' ")
if (is.null(colnames(x2))) stop("give a column names for the 'covariates'")

y <- as.vector(y)
delta <- as.vector(delta)
n <- length(y)

if (!is.null(x21.index) | !is.null(beta2.hat)) {
  x21 <- as.matrix(x2[, x21.index])
  d21 <- ncol(x21)
  x22 <- as.matrix(x2[, -c(1:d21)])
  d22 <- ncol(x22)
  psi.x21 <- as.vector(x21 %*% beta2.hat)
} else {
  psi.x21 <- rep(0, n)
  x22 <- as.matrix(x2)
  d22 <- dim(x22)[2]
}

newx0 <- x22 - t(matrix(rep(x0, d22 * n), d22, n))
ker.h <- as.vector(apply(newx0 / h, MARGIN = 2, FUN = ker, ktype = ktype) / h)
newx <- cbind(1, newx0, newx0^2)

if (is.null(init)) {
  fit.poly <- lm(y[delta == 1] ~ poly(newx0[delta == 1], degree = 2))
  init <- as.numeric(fit.poly$coefficients)
}

if (dist == "tpalnorm") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b2 <- par
    psi.x22 <- as.vector(newx %*% b2)
    psi.x2 <- psi.x21 + psi.x22

    # PH model: time is not rescaled; the hazard is scaled multiplicatively.
    lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp(psi.x2)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b2 <- par
    psi.x22 <- as.vector(newx %*% b2)

```

```

psi.x2 <- psi.x21 + psi.x22

lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x2
chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp(psi.x2)
lhaz0[is.infinite(lhaz0)] <- NA
chaz0[is.infinite(chaz0)] <- NA
val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
return(val)
}
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b2 <- par
    psi.x22 <- as.vector(newx %*% b2)
    psi.x2 <- psi.x21 + psi.x22

    lhaz0 <- htpa(y, eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE) + psi.x2
    chaz0 <- Htpa(y, eta0, phi0, alpha0, F0_La, glog, log = FALSE) * exp(psi.x2)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- OPT$par
return(OUT)
}

#' Local-likelihood estimation of the time-scale smooth term in the AFT
#' model
#' @inheritParams nonparHH_mle
#' @param x design matrix; may combine a parametric block (`x11.index`,
#'   `beta1.hat`) and a non-parametric block
#' @export
nonparAFT_mle <- function(init = NULL, y, delta, x, x11.index = NULL,
                          theta.hat, beta1.hat = NULL, dist,
                          x0, h, ktype = "Epan", method = "Nelder-Mead", maxit = 10000, ...) {
  if (is.na(h) || h <= 0L) stop("invalid 'bandwidth value.' ")
  if (is.null(colnames(x))) stop("give a column names for the 'covariates'")

  y <- as.vector(y)
  delta <- as.vector(delta)
  n <- length(y)

  if (!is.null(x11.index) | !is.null(beta1.hat)) {
    x11 <- as.matrix(x[, x11.index])
    d11 <- ncol(x11)
    x12 <- as.matrix(x[, -c(1:d11)])
  }

```

```

d12 <- ncol(x12)
psi.x11 <- as.vector(x11 %*% beta1.hat)
} else {
psi.x11 <- rep(0, n)
x12 <- as.matrix(x)
d12 <- dim(x12)[2]
}

newx0 <- x12 - t(matrix(rep(x0, d12 * n), d12, n))
ker.h <- as.vector(apply(newx0 / h, MARGIN = 2, FUN = ker, ktype = ktype) / h)
newx <- cbind(1, newx0, newx0^2)

if (is.null(init)) {
fit.poly <- lm(y[delta == 1] ~ poly(newx0[delta == 1], degree = 2))
init <- as.numeric(fit.poly$coefficients)
}

if (dist == "tpalnorm") {
nlog.lik <- function(par) {
if (anyNA(par)) return(Inf)
eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
b1 <- par
psi.x12 <- as.vector(newx %*% b1)
psi.x <- psi.x11 + psi.x12

lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE) + psi.x
chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_N, glog, log = FALSE)
lhaz0[is.infinite(lhaz0)] <- NA
chaz0[is.infinite(chaz0)] <- NA
val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
return(val)
}
}

if (dist == "tpallogis") {
nlog.lik <- function(par) {
if (anyNA(par)) return(Inf)
eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
b1 <- par
psi.x12 <- as.vector(newx %*% b1)
psi.x <- psi.x11 + psi.x12

lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE) + psi.x
chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_Lo, glog, log = FALSE)
lhaz0[is.infinite(lhaz0)] <- NA
chaz0[is.infinite(chaz0)] <- NA
val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
return(val)
}
}

if (dist == "tpalLap") {
nlog.lik <- function(par) {

```

```

    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b1 <- par
    psi.x12 <- as.vector(newx %*% b1)
    psi.x <- psi.x11 + psi.x12

    lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE) + psi.x
    chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_La, glog, log = FALSE)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))
OUT <- OPT$par
return(OUT)
}

#' Local-likelihood estimation of the time-scale smooth term in the AH
#' model
#' @inheritParams nonparAFT_mle
#' @export
nonparAH_mle <- function(init = NULL, y, delta, x, x11.index = NULL,
                        theta.hat, beta1.hat = NULL, dist,
                        x0, h, ktype = "Epan", method = "Nelder-Mead", maxit = 10000, ...) {
  if (is.na(h) || h <= 0L) stop("invalid 'bandwidth value.' ")
  if (is.null(colnames(x))) stop("give a column names for the 'covariates'")

  y <- as.vector(y)
  delta <- as.vector(delta)
  n <- length(y)

  if (!is.null(x11.index) | !is.null(beta1.hat)) {
    x11 <- as.matrix(x[, x11.index])
    d11 <- ncol(x11)
    x12 <- as.matrix(x[, -c(1:d11)])
    d12 <- ncol(x12)
    psi.x11 <- as.vector(x11 %*% beta1.hat)
  } else {
    psi.x11 <- rep(0, n)
    x12 <- as.matrix(x)
    d12 <- dim(x12)[2]
  }

  newx0 <- x12 - t(matrix(rep(x0, d12 * n), d12, n))
  ker.h <- as.vector(apply(newx0 / h, MARGIN = 2, FUN = ker, ktype = ktype) / h)
  newx <- cbind(1, newx0, newx0^2)

  if (is.null(init)) {
    fit.poly <- lm(y[delta == 1] ~ poly(newx0[delta == 1], degree = 2))
    init <- as.numeric(fit.poly$coefficients)
  }
}

```



```

}

if (dist == "tpalnorm") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b1 <- par
    psi.x12 <- as.vector(newx %*% b1)
    psi.x <- psi.x11 + psi.x12

    # AH model: time rescaled by exp(psi.x), cumulative hazard divided
    # by exp(psi.x) (i.e. multiplied by exp(-psi.x)).
    lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_N, F0_N, glog, log = TRUE)
    chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_N, glog, log = FALSE) * exp(-psi.x)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpallogis") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b1 <- par
    psi.x12 <- as.vector(newx %*% b1)
    psi.x <- psi.x11 + psi.x12

    lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_Lo, F0_Lo, glog, log = TRUE)
    chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_Lo, glog, log = FALSE) * exp(-psi.x)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

if (dist == "tpalLap") {
  nlog.lik <- function(par) {
    if (anyNA(par)) return(Inf)
    eta0 <- exp(theta.hat[1]); phi0 <- exp(theta.hat[2]); alpha0 <- expit(theta.hat[3])
    b1 <- par
    psi.x12 <- as.vector(newx %*% b1)
    psi.x <- psi.x11 + psi.x12

    lhaz0 <- htpa(y * exp(psi.x), eta0, phi0, alpha0, f0_La, F0_La, glog, log = TRUE)
    chaz0 <- Htpa(y * exp(psi.x), eta0, phi0, alpha0, F0_La, glog, log = FALSE) * exp(-psi.x)
    lhaz0[is.infinite(lhaz0)] <- NA
    chaz0[is.infinite(chaz0)] <- NA
    val <- -sum(delta * lhaz0 * ker.h, na.rm = TRUE) + sum(chaz0 * ker.h, na.rm = TRUE)
    return(val)
  }
}

```

```
}  
  
if (method != "nlminb") OPT <- optim(init, nlog.lik, control = list(maxit = maxit), method = method)  
if (method == "nlminb") OPT <- nlminb(init, nlog.lik, control = list(iter.max = maxit))  
OUT <- OPT$par  
return(OUT)  
}
```

6 Post-Estimation Tools

```

#' Average squared error between two survival curves
#' @param true_sp true/reference survival probabilities
#' @param pred_sp predicted survival probabilities
#' @return a single numeric value (the mean squared difference)
#' @export
surv_ase <- function(true_sp, pred_sp) {
  if (length(true_sp) != length(pred_sp))
    stop(" Error: surv_sp and pred_sp must have equal length")
  d <- (true_sp - pred_sp)^2
  return(ase = mean(d, na.rm = TRUE))
}

#' Predicted survival, hazard, cumulative hazard, or quantile from a fitted
#' TPA (or "EW") model
#'
#' @param tau probability(-ies) for `type = "quantile"` (in (0,1))
#' @param theta transformed baseline parameters `c(log(eta), log(phi),
#'   logit(alpha))` -- NOTE: the function body actually reads these from an
#'   object called `par`, not from the `theta` argument (see the "Known
#'   limitations" note in Section 9); supply a global/enclosing `par`
#'   vector with the natural-scale baseline parameters `c(eta, phi, alpha)`
#'   until this is corrected
#' @param time vector of times at which to predict
#' @param psihat.1 fitted time-scale linear predictor (vector, length n)
#' @param psihat.2 fitted hazard-scale linear predictor (vector, length n)
#' @param dist baseline distribution: "tpalnorm", "tpallogis", or "EW"
#'   (Exponentiated Weibull -- requires a package providing `pexpweibull`/
#'   `hexpweibull`/`chexpweibull`/`qexpweibull`, which is NOT loaded
#'   anywhere in this file; see Section 9)
#' @param type one of "survival" (default), "hazard", "cmhazard", "quantile"
#' @return numeric vector of predictions, one per (sorted) time / tau value
#' @export
predict_fun <- function(tau = NULL, theta, time, psihat.1, psihat.2,
                        dist, type = "survival", ...) {
  time <- sort(time)
  psihat.dif <- psihat.2 - psihat.1

  if (type == "survival") {
    if (dist == "tpalnorm") {
      value <- ptpa(time * exp(psihat.1), par[1], par[2], par[3],
                    FO_N, glog, lower.tail = FALSE)^exp(psihat.dif)
    } else if (dist == "tpallogis") {
      value <- ptpa(time * exp(psihat.1), par[1], par[2], par[3],
                    FO_Lo, glog, lower.tail = FALSE)^exp(psihat.dif)
    } else if (dist == "EW") {
      value <- 1 - pexpweibull(time * exp(psihat.1), par[1], par[2], par[3])^exp(psihat.dif)
    } else stop("Baseline distribution is not provided")
  } else if (type == "hazard") {
    if (dist == "tpalnorm") {
      value <- htpa(time * exp(psihat.1), par[1], par[2], par[3],
                    f0_N, FO_N, glog) * exp(psihat.2)
    } else if (dist == "tpallogis") {

```

```

    value <- htpa(time * exp(psihat.1), par[1], par[2], par[3],
                  f0_Lo, F0_Lo, glog) * exp(psihat.2)
  } else if (dist == "EW") {
    value <- hexpweibull(time * exp(psihat.1), par[1], par[2], par[3]) * exp(psihat.2)
  } else stop("Baseline distribution is not provided")
} else if (type == "cmhazard") {
  if (dist == "tpalnorm") {
    value <- Htpa(time * exp(psihat.1), par[1], par[2], par[3], F0_N, glog) * exp(psihat.dif)
  } else if (dist == "tpallogis") {
    value <- Htpa(time * exp(psihat.1), par[1], par[2], par[3], F0_Lo, glog) * exp(psihat.dif)
  } else if (dist == "EW") {
    value <- chexpweibull(time * exp(psihat.1), par[1], par[2], par[3]) * exp(psihat.dif)
  } else stop("Baseline distribution is not provided")
} else if (type == "quantile") {
  if (dist == "tpalnorm") {
    value <- exp(-psihat.1) * qtpa(1 - exp(log(1 - tau) / exp(psihat.dif)), par[1], par[2], par[3],
                                   F0_N, glog, glog.inv, QF_N)
  } else if (dist == "tpallogis") {
    value <- exp(-psihat.1) * qtpa(1 - exp(log(1 - tau) / exp(psihat.dif)), par[1], par[2], par[3],
                                   F0_Lo, glog, glog.inv, QF_Lo)
  } else if (dist == "EW") {
    value <- exp(-psihat.1) * qexpweibull(1 - exp(log(1 - tau) / exp(psihat.dif)), par[1], par[2], par[3])
  } else stop("Baseline distribution is not provided")
}
return(value)
}

```

```

#' Simple scatter/line plot of a predicted quantity against time
#' @param object predicted values (e.g. output of predict_fun)
#' @param time the corresponding times
#' @param xlab,ylab axis labels
#' @export
predict_plot <- function(object, time, xlab = "Time", ylab = "Survival probability") {
  ord <- order(time)
  plot(time[ord], object[ord], xlab = xlab, ylab = ylab)
}

```

```

#' Numerical integration of an estimated derivative curve via the
#' trapezoidal rule, to recover  $\psi(x)$  from an estimate of  $\psi'(x)$ 
#'
#' Two implementations are kept, matching the original script:
#' `trap_fun()` uses `pracma::trapz()`, `trap_phihat()` is a
#' dependency-free hand-written equivalent that additionally requires  $x0$ 
#' to already be sorted.
#'
#' @param x0 vector of grid values (the points at which the derivative was
#' estimated)
#' @param dpsi.hat estimated derivative of  $\psi$  at each  $x0$ 
#' @return cumulative (trapezoidal) integral of dpsi.hat from the smallest
#'  $x0$  up to each grid point, i.e. an estimate of  $\psi(x0)$ 
#' @export
trap_fun <- function(x0, dpsi.hat) {
  require(pracma)
  if (is.unsorted(x0)) {

```

```

    ord <- order(x0)
    x0 <- x0[ord]
    dpsi.hat <- dpsi.hat[ord]
  }

  m <- length(x0)
  psi.hat <- as.null(x0)
  psi.hat[1] <- (x0[1] / 2) * dpsi.hat[1]
  for (k in 2:m) {
    s.x <- sapply(2:k, function(j) trapz(x = x0[(j - 1):j], y = dpsi.hat[(j - 1):j]))
    psi.hat[k] <- sum(s.x)
  }
  return(psi.hat = as.numeric(psi.hat))
}

#' @rdname trap_fun
#' @export
trap_phihat <- function(x0, dpsi.hat) {
  if (!is.sorted(x0))
    stop("'x0' should be sorted")
  m <- length(x0)
  psi.hat <- as.null(x0)
  psi.hat[1] <- (x0[1] / 2) * dpsi.hat[1]
  for (k in 2:m) {
    s.x <- sapply(2:k, function(j) (x0[j] - x0[j - 1]) * (dpsi.hat[j] + dpsi.hat[j - 1]) / 2)
    psi.hat[k] <- sum(s.x)
  }
  return(psi.hat = as.numeric(psi.hat))
}

#' Wald-type confidence intervals via a finite-difference Hessian
#'
#' Computes the observed Fisher information as the inverse of a
#' numerically approximated Hessian of the negative log-likelihood
#' (via `nlme::fdHess()`), then builds symmetric Wald confidence intervals
#' around the MLE.
#'
#' @param nlogLik the negative log-likelihood function (e.g. the
#'   `nloglik` element returned by any `par*_mle()` function)
#' @param mle the maximum-likelihood estimate (vector), i.e. `OPT$par` (or
#'   `OPT$OPT$par` from the `par*_mle()` output list)
#' @param level confidence level, default 0.95
#' @return a matrix with columns Estimates, Std. Error, Lower, Upper; one
#'   row per parameter
#' @export
Conf.Int.FD <- function(nlogLik, mle, level = 0.95, ...) {
  require(nlme)
  sd.int <- abs(qnorm(0.5 * (1 - level)))
  HESS <- fdHess(pars = mle, fun = nlogLik)
  Fisher.Info <- solve(HESS$Hessian)
  se <- sqrt(diag(Fisher.Info))
  U <- mle + sd.int * se
  L <- mle - sd.int * se
  C.I <- cbind(mle, se, L, U)

```

```
names.row <- paste0("par", seq_along(1:length(mle)))
rownames(C.I) <- names.row
colnames(C.I) <- c("Estimates", "Std. Error", "Lower", "Upper")
return(C.I)
}
```

7 Worked Example

This example simulates right-censored survival data from a **TPA-normal Proportional Hazards** model with a single binary covariate, fits the model with `parPH_mle()`, obtains standard errors and 95% confidence intervals with `Conf.Int.FD()`, and predicts/plots the fitted survival curve for each group using `htpa()/ptpa()` directly (a thin, corrected stand-in for `predict_fun()` – see the note on `predict_fun()` in Section 9).

```
set.seed(2026)
n <- 300

# One binary covariate (e.g. treatment arm).
x <- matrix(rbinom(n, 1, 0.5), ncol = 1)
colnames(x) <- "trt"

# True (data-generating) parameters: baseline TPA-normal parameters and a
# single hazard-scale (PH) log-hazard-ratio.
eta_true <- 1.2
phi_true <- 0.6
alpha_true <- 0.4
beta_true <- -0.7 # trt = 1 halves the hazard, roughly

# Simulate event times: the PH construction used in parPH_mle() rescales
# only the hazard, not time, so we invert the model's own survival
# function group-by-group using the quantile function qtpa().
u <- runif(n)
psi.x <- as.vector(x %*% beta_true)
# Under this PH construction,  $S(t | x) = S_0(t)^{\exp(\psi.x)}$ , so inverting:
#  $u = S_0(t)^{\exp(\psi.x)} \Rightarrow S_0(t) = u^{(1/\exp(\psi.x))} \Rightarrow t = QF_0(1 - S_0(t))$ 
target_tau <- 1 - u^(1 / exp(psi.x))
true_time <- sapply(target_tau, function(p) {
  qtpa(p, eta_true, phi_true, alpha_true, F0_N, glog, glog.inv, QF_N)
})

# Independent random censoring.
cens_time <- rexp(n, rate = 0.15)
y <- pmin(true_time, cens_time)
delta <- as.numeric(true_time <= cens_time)

cat("Simulated n =", n, " | events =", sum(delta), " | censored =", sum(1 - delta), "\n")

## Simulated n = 300 | events = 177 | censored = 123

# Starting values: log(eta), log(phi), logit(alpha), beta.
init <- c(log(1), log(1), 0, 0)

fit <- parPH_mle(init = init, y = y, delta = delta, x = x,
  dist = "tpalnorm", method = "nlminb", maxit = 5000)

est <- fit$OPT$par
names(est) <- c("log(eta)", "log(phi)", "logit(alpha)", "beta_trt")
knitr::kable(round(cbind(Estimate = est), 4),
  caption = "MLE on the transformed (unconstrained) scale")
```

Table 3: MLE on the transformed (unconstrained) scale

	Estimate
log(eta)	0.2431
log(phi)	-0.6070
logit(alpha)	-0.4610
beta_trt	-0.5573

```
# Back-transform to the natural scale for interpretation.
natural <- c(eta = exp(est[1]), phi = exp(est[2]), alpha = expit(est[3]),
            hazard_ratio_trt = exp(est[4]))
knitr::kable(round(cbind(Estimate = natural), 4),
              caption = "Natural-scale parameters (hazard_ratio_trt is exp(beta_trt))")
```

Table 4: Natural-scale parameters (hazard_ratio_trt is exp(beta_trt))

	Estimate
eta.log(eta)	1.2752
phi.log(phi)	0.5450
alpha.logit(alpha)	0.3868
hazard_ratio_trt.beta_trt	0.5727

```
ci <- Conf.Int.FD(nlogLik = fit$nloglik, mle = fit$OPT$par)
rownames(ci) <- c("log(eta)", "log(phi)", "logit(alpha)", "beta_trt")
knitr::kable(round(ci, 4),
              caption = "Wald 95% confidence intervals (transformed scale)")
```

Table 5: Wald 95% confidence intervals (transformed scale)

	Estimates	Std. Error	Lower	Upper
log(eta)	0.2431	0.2331	-0.2137	0.7000
log(phi)	-0.6070	0.0721	-0.7483	-0.4658
logit(alpha)	-0.4610	0.2664	-0.9832	0.0612
beta_trt	-0.5573	0.1525	-0.8562	-0.2585

```
eta_hat <- exp(est[1]); phi_hat <- exp(est[2]); alpha_hat <- expit(est[3])
beta_hat <- est[4]

tt <- seq(0.01, quantile(y, 0.95), length.out = 200)

# PH model: S(t | x) = S0(t)^exp(x * beta_hat)
S0 <- ptpa(tt, eta_hat, phi_hat, alpha_hat, F0_N, glog, lower.tail = FALSE)
S_trt0 <- S0^exp(0 * beta_hat) # trt = 0
S_trt1 <- S0^exp(1 * beta_hat) # trt = 1

plot(tt, S_trt0, type = "l", lwd = 2, ylim = c(0, 1),
      xlab = "Time", ylab = "Survival probability",
      main = "Fitted survival curves: TPA-normal PH model")
lines(tt, S_trt1, lwd = 2, lty = 2, col = "steelblue")
```



```
legend("topright", legend = c("trt = 0", "trt = 1"),
      lwd = 2, lty = c(1, 2), col = c("black", "steelblue"), bty = "n")
```

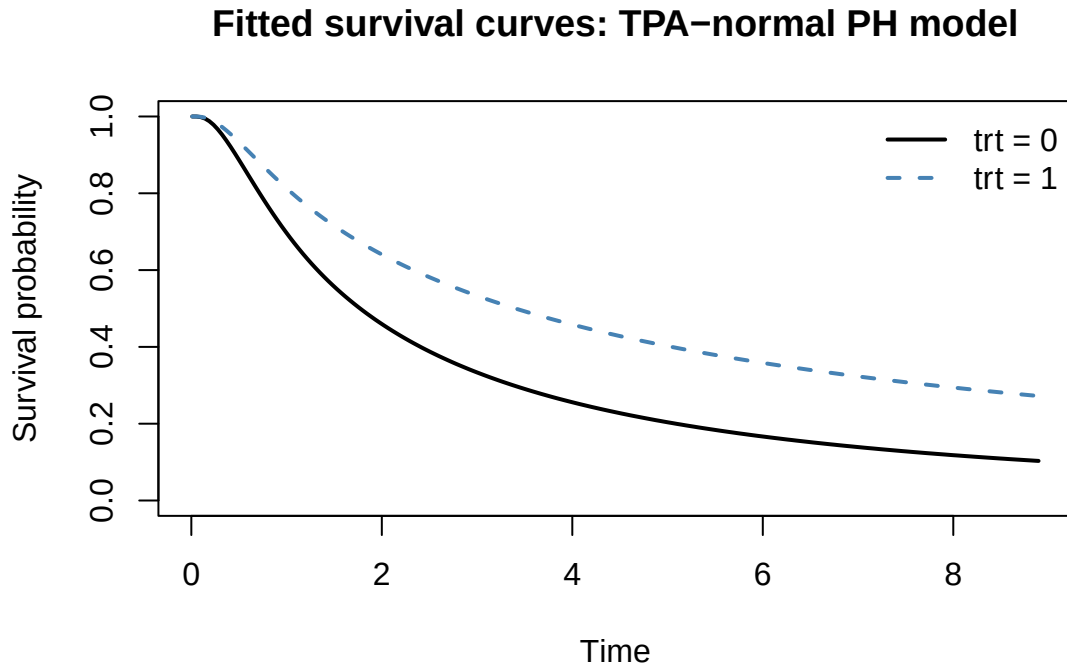


Figure 2: Fitted TPA-normal PH survival curves by treatment group

The fitted `beta_trt` should recover a value in the neighbourhood of the true -0.7 used to simulate the data, subject to sampling variability at `n = 300`; the printed 95% confidence interval for `beta_trt` should typically cover -0.7.

8 Function Reference Table

Table 6: One-line summary of every function in this file

Function	Purpose
f0_N/F0_N/QF_N	Normal reference density/CDF/quantile
f0_La/F0_La/QF_La	Laplace reference density/CDF/quantile
f0_Lo/F0_Lo/QF_Lo	Logistic reference density/CDF/quantile
glog/glog.inv, glogit/glogit.inv	Link functions for the time scale
dtpa	TPA family density
ptpa	TPA family CDF / survival function
htpa	TPA family hazard function
Htpa	TPA family cumulative hazard function
qtpa	TPA family quantile function
rtpa	Random draws from the TPA family
parHH_mle	Parametric MLE, Hybrid Hazard model
parGH_mle	Parametric MLE, General Hazards model
parAFT_mle	Parametric MLE, Accelerated Failure Time model
parPH_mle	Parametric MLE, Proportional Hazards model
parAH_mle	Parametric MLE, Accelerated Hazards model
ker	Kernel weight functions
nonparHH_mle	Local-likelihood MLE, HH model (hazard-scale smooth term)
nonparPH_mle	Local-likelihood MLE, PH model (hazard-scale smooth term)
nonparAFT_mle	Local-likelihood MLE, AFT model (time-scale smooth term)
nonparAH_mle	Local-likelihood MLE, AH model (time-scale smooth term)
surv_ase	Average squared error between two survival curves
predict_fun	Predicted survival/hazard/cum.hazard/quantile (see Sec. 9 note)
predict_plot	Simple plot of predicted quantity vs. time
trap_fun / trap_phihat	Trapezoidal-rule numerical integration of an estimated derivative
Conf.Int.FD	Finite-difference Wald confidence intervals

9 Notes on the Original Code

These notes are included so that users of this rewritten file know exactly what was inherited as-is from the original `Topic3_sourcecode.R`, without any change to behaviour. Nothing in the executable code above was modified relative to the original script (only comments, spacing, and file organisation changed); the two items below are pointed out so you can decide whether to work around them or patch them yourself.

1. **`predict_fun()` reads baseline parameters from a variable called `par`, not from its own `theta` argument.** In the original script the function signature includes a `theta` parameter that is documented but never used inside the function body; instead, every branch references an object literally called `par`. This works only if a suitably named vector `par` (natural-scale `c(eta, phi, alpha)`) happens to exist in the calling environment (for example, left over from a prior `optim()` call that used `par` as its variable name). In the worked example above we avoided `predict_fun()` for this reason and instead called `ptpa()/htpa()/Htpa()` directly with the fitted parameters. If you want to use `predict_fun()` as-is, define a variable named exactly `par <- c(eta_hat, phi_hat, alpha_hat)` before calling it.
2. **The `dist = "EW"` branches of `predict_fun()` call functions that are not defined or imported anywhere in this file** (`pexpweibull`, `hexpweibull`, `chexpweibull`, `qexpweibull` — these come from an Exponentiated-Weibull package that the original script never loads). These branches will fail with an “object not found” error unless you load a package that provides those four functions yourself.
3. **`nonparHH_mle()` and `nonparPH_mle()` implement `dist = "tpalnorm"` and `"tpallogis"` (and, for `nonparPH_mle()`, `"tpallap"`), but do not implement a TPA log-Laplace branch for `nonparHH_mle()`.** Calling `nonparHH_mle(..., dist = "tpallap")` will error because `nlog.lik` is never assigned. This mirrors the original script exactly.
4. **`trap_phihat()` calls `is.sorted()`**, which is not a base-R function (base R provides `is.unsorted()`, whose negation is what’s presumably intended). This function will error immediately if called, in both the original script and this rewrite. Prefer `trap_fun()` (which uses `pracma::trapz()` and handles unsorted input automatically) unless you first fix this line yourself.
5. **All optimisers use `method = "Nelder-Mead"` by default**, a derivative-free method that is robust to a poorly chosen starting point but can be slow to converge for larger parameter vectors; for production use, consider trying `method = "nllminb"` or `method = "BFGS"` as well and comparing results, especially once you add more than a handful of covariates.

10 Original Repository

- Source: <https://github.com/Ewnetu-github/tpaHH>
- Original single file: `Topic3_sourcecode.R`
- Original README: *“This is a source code for a hybrid hazard-based models using two-piece distributions. We proposed parametric, semi-parametric and non-parametric regression functions for the time-scale and relative hazard-scale change. Three parametric baseline distributions with a log-link function are included in the code: TPA normal, TPA logistic and TPA Laplace distributions.”*

This document (`tpaHH_two_piece_hazard_models.Rmd`) can be re-knitted at any time with:

```
rmarkdown::render("tpaHH_two_piece_hazard_models.Rmd", output_format = "pdf_document")
rmarkdown::render("tpaHH_two_piece_hazard_models.Rmd", output_format = "word_document")
```